

PROJECT REPORT

AKSHAT SRIAVASTAVA
25BAI11109 (VITYARTHI PROJECT)

- Student → Add topic
- Student → Add dependency
- Student → View dependency graph
- Student → Generate study plan
- Student → Update progress
- Student → View progress
- Student → Save/load data

6.3 Class / Component Diagram — textual representation

Main → SyllabusGraph → Topic. Main → Scheduler implementations. Scheduler → StudyPlan → StudySession. Main → ProgressTracker. Main → PersistenceManager. StudyForgeException provides a custom exception type.

6.4 Sequence Diagram — study-plan generation

Student → Main: choose Generate Study Plan → Main → Scheduler: generate(...) → Scheduler → SyllabusGraph: obtain topics/prerequisites → Scheduler: filter and prioritize available topics → Scheduler → StudyPlan: create sessions → Main → Student: print plan.

6.5 Storage Design

The current implementation uses file-based persistence rather than a relational database. PersistenceManager stores topic and dependency information in a text file, so an ER diagram is not applicable.

7. Design Decisions & Rationale

- Adjacency lists represent topic relationships efficiently and support graph traversal.
- DFS cycle detection prevents circular prerequisite relationships.
- Kahn's algorithm creates an order that respects prerequisites.
- GreedyScheduler and CPMScheduler provide selectable heuristic scheduling strategies.
- File-based persistence keeps the current implementation lightweight while demonstrating storage.
- Package separation improves maintainability and makes individual components easier to test.

8. Implementation Details

The model package contains Status, Topic, StudySession, and StudyPlan. Topic stores identity, name, difficulty, estimated hours, deadline, and progress status.

SyllabusGraph maintains topics and an adjacency-list representation. Dependency insertion validates self-cycles and potential circular relationships. The graph also provides prerequisite lookup and topological sorting.

Scheduler is an interface implemented by GreedyScheduler and CPMScheduler. GreedyScheduler prioritizes available topics by deadline, then difficulty and name. CPMScheduler computes a downstream workload weight and prioritizes topics with larger downstream impact, followed by deadline and name.

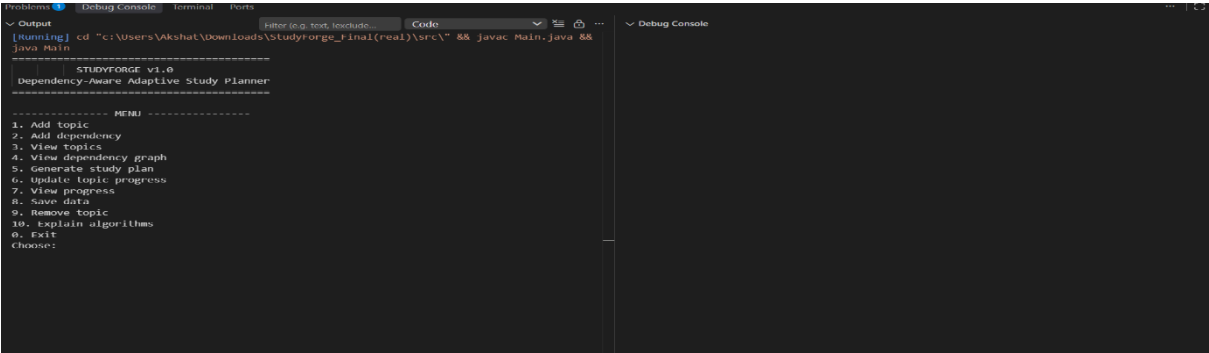
ProgressTracker updates status and calculates completion percentage. PersistenceManager supports file-based save/load. Main.java exposes these features through the interactive menu.

The scheduling approaches are heuristic/priority-based and should not be described as mathematically optimal for every possible syllabus.

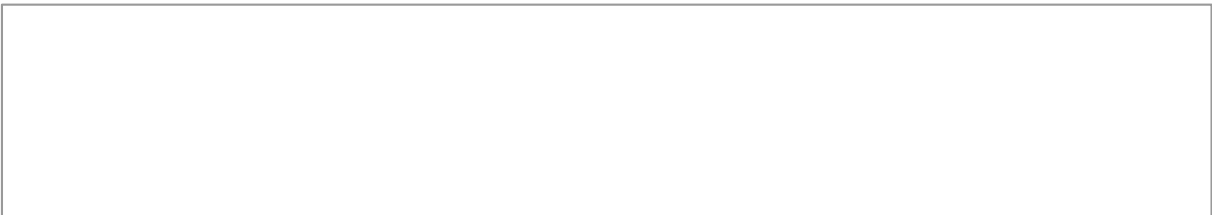
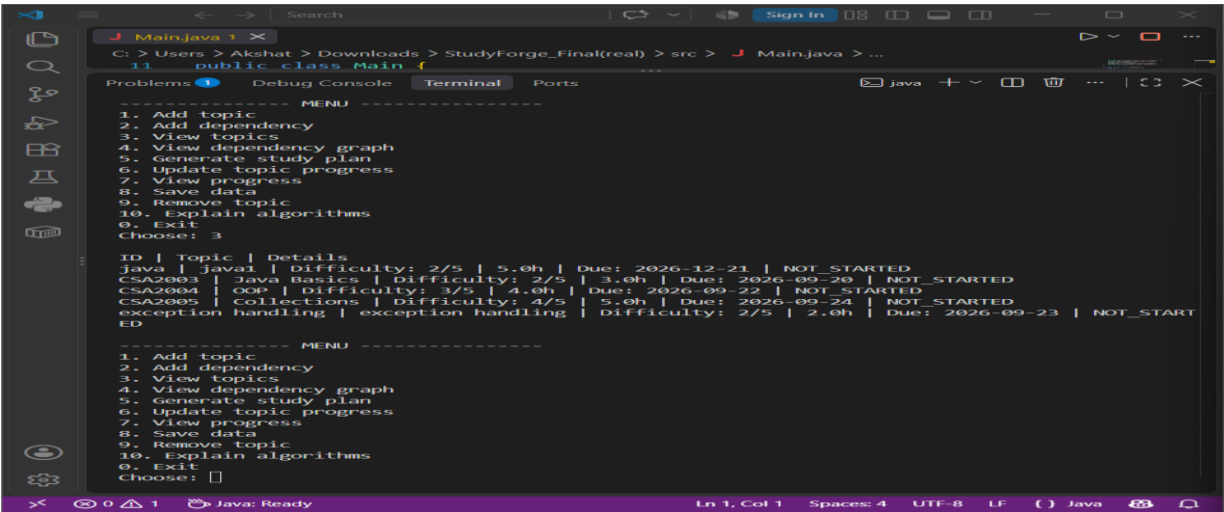
9. Screenshots / Results

Insert actual screenshots from the final application demonstration in the placeholders below. Recommended captures are the main menu, topic/dependency setup, generated plan, progress update, and adaptive regenerated plan.

Main Menu



Topics + Dependency Graph



Generated Study Plan

```
Using: Greedy Deadline-First Scheduler
2026-09-18:
-> OOP (3.0h)
2026-09-19:
-> OOP (1.0h)
-> exception handling (2.0h)
2026-09-20:
-> Collections (3.0h)
2026-09-21:
-> Collections (2.0h)
-> java1 (1.0h)
2026-09-22:
-> java1 (3.0h)
2026-09-23:
-> java1 (1.0h)
Tip: If progress changes, regenerate the plan to adapt.
----- MENU -----
1. Add topic
2. Add dependency
3. View topics
4. View dependency graph
5. Generate study plan
6. Update topic progress
7. View progress
8. Save data
9. Remove topic
```

Progress Update

INSERT APPLICATION SCREENSHOT HERE

```
----- MENU -----
1. Add topic
2. Add dependency
3. View topics
4. View dependency graph
5. Generate study plan
6. Update topic progress
7. View progress
8. Save data
9. Remove topic
10. Explain algorithms
0. Exit
Choose: 6
Topic ID: CSA2003
1. NOT_STARTED
2. IN_PROGRESS
3. COMPLETED
4. SKIPPED
Status: 3
Progress updated. Generate a new plan to recalculate the remaining schedule.
----- MENU -----
1. Add topic
2. Add dependency
3. View topics
4. View dependency graph
5. Generate study plan
6. Update topic progress
7. View progress
8. Save data
9. Remove topic
10. Explain algorithms
0. Exit
Choose: 
```



```
1. Main.java
C:\> Users > Akshat > Downloads > StudyForge_Final(real) > src > Main.java > ...
1. public class Main {
    9. Remove topic
    10. Explain algorithms
    0. Exit
    Choose: 5
    Available study hours per day: 3
    Start date (YYYY-MM-DD): 2026-09-18
    1. Greedy Deadline-First
    2. Critical Path Weighted
    Choose scheduler: 1
    Using: Greedy Deadline-First scheduler
    2026-09-18:
    -> OOP (3.0h)
    2026-09-19:
    -> OOP (1.0h)
    -> exception handling (2.0h)
    2026-09-20:
    -> Collections (3.0h)
    2026-09-21:
    -> Collections (2.0h)
    -> java1 (1.0h)
    2026-09-22:
    -> java1 (3.0h)
    2026-09-23:
    -> java1 (1.0h)
    Tip: If progress changes, regenerate the plan to adapt.
}
```

10. Testing Approach

StudyForge includes StudyForgeTest.java. The documented procedure compiles the project and test class and runs the test class with Java assertions enabled. The expected successful output is “ALL TESTS PASSED”.

Test Area	Expected behavior
Topological sort	Produces an order respecting prerequisite relationships.
Cycle detection	Detects circular dependencies.
Cycle rejection	Rejects a dependency that would introduce a cycle.
Greedy scheduling	Generates a dependency-valid schedule.
Completed-topic exclusion	Completed topics do not appear in regenerated workload.

Documented test commands:

```
javac -d out src\model\*.java src\graph\*.java src\algorithm\*.java src\service\*.java src\persistence\*.java src\exception\*.java tests\StudyForgeTest.java
```

```
java -ea -cp out StudyForgeTest
```

Expected output: ALL TESTS PASSED

11. Challenges Faced

- Representing prerequisite relationships as a graph requires explicit handling of invalid and circular dependencies.
- Scheduling must respect dependency availability while considering workload and deadlines.
- Adaptive scheduling requires progress state to affect which topics remain eligible.
- Maintaining a clean multi-package Java structure requires clear separation of responsibilities.
- Testing must cover both normal behavior and invalid dependency cases.

12. Learnings & Key Takeaways

- Applied directed graphs and adjacency lists to a practical dependency problem.
- Applied DFS cycle detection and Kahn's topological sorting.
- Used interfaces and multiple implementations for interchangeable scheduling strategies.
- Used Java collections and the Java Time API.
- Practiced validation, modular design, automated tests, documentation, and Git-based organization.
- Distinguished heuristic scheduling from mathematical optimization.

13. Future Enhancements

- Add a graphical or web-based interface.
- Support fixed study sessions, breaks, weekends, and unavailable hours.
- Add more scheduling strategies and comparison views.
- Add workload distribution and deadline-risk analytics.
- Replace file storage with a database for larger or multi-user deployments.
- Add schedule export functionality.

14. References

- VITyarthi — Build Your Own Project: General Project Instructions & Submission Guidelines.
- StudyForge README.md — project overview, requirements, compile/run/test procedure, and algorithmic concepts.
- StudyForge statement.md — problem statement, scope, target users, and high-level features.
- StudyForge source code under src/.
- StudyForgeTest.java — automated validation of graph and scheduling behavior.

15. Submission Readiness Checklist

Requirement	Status
Original, course-relevant project	Prepared
3+ functional modules	Prepared
4+ non-functional requirements	Prepared
5–10+ meaningful classes/files	Prepared
Proper package/folder structure	Prepared
Testing	Prepared
README.md	Prepared
statement.md	Prepared
Architecture and workflow	Prepared
UML/design documentation	Prepared
15-section report	Prepared
Screenshots/results	Insert final application screenshots
GitHub repository	Created and organized

Before portal submission, replace the screenshot placeholders with actual application captures and perform one final check of the GitHub repository contents.